



安徽大學
人工智能學院
School of Artificial Intelligence
Anhui University

《机器人语音信息处理》报告

基于隐马尔可夫模型的孤立字语音识别系统

学 院: 人工智能学院

专 业: 机器人工程

学 号: WA2224013

姓 名: 郭义月

指导老师: 杨普海

课程编号: ZX52034

课程学分: 2

提交日期: 2025.5.15

目 录

摘要	3
1 理论介绍	4
1.1 语音采集方法	4
1.1.1 语音录制	4
1.1.2 语音读取	4
1.1.3 语音保存	4
1.2 端点检测方法	5
1.2.1 双门限法的原理	5
1.2.2 双门限法的实现	5
1.3 训练 HMM 的过程及其中涉及的三大算法	7
1.3.1 前向-后向算法	7
1.3.2 维特比 Viterbi 算法	9
1.3.3 Baum-Welch 算法	9
2 数据采集与预处理	12
2.1 语音采集	12
2.2 端点检测	13
2.3 特征提取	14
2.3.1 MFCC 特征提取方法介绍	14
2.3.2 代码实现	15
3 实验分析	17
3.1 多说话人实验	17
3.1.1 多说话人的 HMM 训练	17
3.1.2 多说话人的 HMM 测试	18
3.2 单说话人实验	20
3.2.1 单说话人的 HMM 训练	20
3.2.2 单说话人的 HMM 测试	20
3.3 HMM 状态数的影响	21
3.4 高斯混合成分数的影响	23
4 总结	24
A 附录：代码	27
B 附录：端点检测部分的参数设置	31

摘要

本课程设计旨在搭建一个基于隐马尔可夫模型 (HMM) 的孤立字语音识别系统, 以深入理解语音信息处理的关键技术和经典算法。通过完成语音采集、预处理、模型训练、模型测试以及参数分析等环节, 系统地探索语音识别技术的实际应用。在语音采集阶段, 依据《语音信号处理实验教程》, 成功采集了来自 3 个人的 10 个中文数字 (0 到 9) 的语音数据, 并完成了语音文件的存储与查看。预处理及模型训练过程中, 采用了端点检测技术去除静音段, 用 MFCC 算法提取了语音信号的特征参数, 并利用 HMM 进行模型训练。在模型测试环节, 通过对比单说话人和多说话人实验, 分析了不同说话人的语音对识别结果的影响。此外, 还针对 HMM 状态数和高斯混合成分数进行了实验分析, 探讨了其对识别性能的影响。实验结果表明, 所搭建的孤立字语音识别系统在单说话人情况下表现较为理想, 合理调整参数可达到 100% 的准确率, 但是在多说话人情况下, 表现则并不好。本设计也探讨了 HMM 状态数和高斯混合成分数的调整对识别性能和模型复杂度的影响。通过本次课程设计, 进一步加深了对语音信息处理相关知识和技术的理解, 为后续的语音识别研究和应用开发奠定了基础。

本实验的流程图如图12所示:

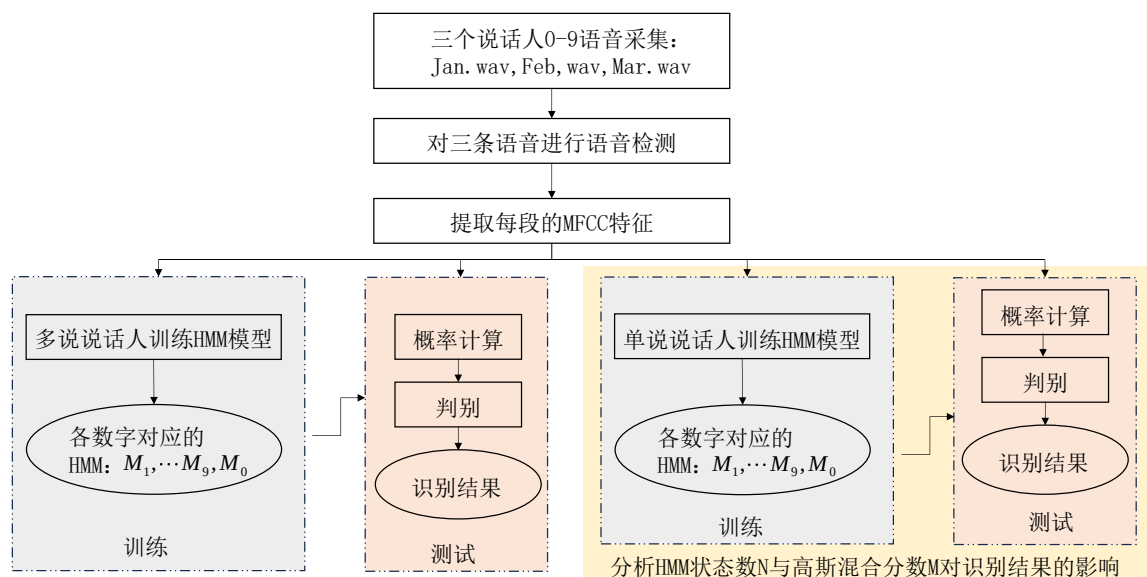


图 1: 本实验的流程图

1 理论介绍

1.1 语音采集方法

为了将原始模拟语音信号变为数字信号，必须经过采样和量化两个步骤，从而得到时间和幅度上均为离散的数字信号语音。采样是信号在时间上的离散化，即按照一定时间间隔 Δt 在模拟信号 $x(t)$ 上逐点采取其瞬时值。采样时必须要注意满足奈奎斯特定理，即采样频率 f 必须以高于被测信号的最高频率两倍以上速度进行取样，才能正确地重建信号¹。

在 Windows 环境下，可以使用 Windows 自带的录音机录制语音文件，声卡可以完成语音波形的 A/D 转换，获得 WAV 文件。通过 Windows 录制的语音信号，一方面可以为后续实验储备原始语音，另一方面可以与通过其他方式录制的语音进行比对。

需要注意：参考书上的为 MATLAB 较早期版本的用法，在新的版本中，以及无法识别相关的函数。在 MATLAB 的早期版本中，语音信号的录制和处理主要依赖于 wavrecord、wavread 和 wavwrite 等函数。然而，从 MATLAB 2015 版本开始，这些函数已经被更新或替代，以提供更强大的功能和更好的兼容性²。以下是这些更新的具体内容：

1.1.1 语音录制

在 MATLAB 早期版本中，wavrecord 函数用于录制语音信号。然而，从 MATLAB R2015a 开始，wavrecord 函数已被废弃，取而代之的是 audiorecorder 函数。audiorecorder 提供了更灵活的录音功能，支持多通道录音、不同的采样率和采样位数。

创建录音对象：其中，Fs 是采样率，nBits 是采样位数（8, 16, 32 可选），nChannels 是通道数（1, 2 可选）

```
recorder = audiorecorder(Fs, nBits, nChannels);  
recordblocking(recorder, during); %开始录音，during是录制时长
```

recordblocking 得到的 recorder 是一个 1*1 的 audiorecorder 对象。可以通过 getaudiodata 将录制的音频信号存储在数值数组中。

```
audioData = getaudiodata(recorder);
```

1.1.2 语音读取

在 MATLAB 早期版本中，wavread 函数用于读取 WAV 文件中的语音信号。从 MATLAB R2015a 开始，wavread 函数已被废弃，推荐使用 audioread 函数。audioread 函数不仅支持 WAV 文件，还支持多种音频格式，如 MP3、FLAC 等。

```
[audioData, Fs] = audioread('test.wav');  
player = audioplayer(audioData, Fs); % 播放音频  
play(player);
```

1.1.3 语音保存

在 MATLAB 早期版本中，wavwrite 函数用于将语音信号保存为 WAV 文件。从 MATLAB R2015a 开始，wavwrite 函数已被废弃，推荐使用 audiowrite 函数。audiowrite 函数提供了更灵活的保存选项，支持多种音频格式和编码方式。

```
audiowrite('recorded_audio.wav', audioData, Fs);
```

1.2 端点检测方法

参考书上介绍了很多端点检测的方法，例如双门限法，相关法，比例法，熵谱法等。在本设计中，会详细介绍双门限法的原理和实现。

1.2.1 双门限法的原理

语音端点检测本质上是根据语音和噪声的相同参数所表现出的不同特征来进行区分。传统的短时能量和过零率相结合的语音端点检测算法利用短时过零率来检测清音，用短时能量来检测浊音，两者相配合便实现了信号信噪比较大情况下的端点检测。算法以短时能量检测为主，短时过零率检测为辅。根据语音的统计特性，可以把语音段分为清音、浊音以及静音（包括背景噪声）三种。

(1) 短时能量

设第 n 帧语音信号 $x_n(m)$ 的短时能量用 E_n 表示，则其计算公式如下：

$$E_n = \sum_{m=0}^{N-1} x_n^2(m) \quad (1)$$

E_n 是一个度量语音信号幅度值变化的函数，但它有一个缺陷，即对高电平非常敏感（因为它计算时用的是信号的平方）。

(2) 短时过零率

短时过零率表示一帧语音中语音信号波形穿过横轴（零电平）的次数。对于连续语音信号，过零即意味着时域波形通过时间轴；而对于离散信号，如果相邻的取样值改变符号则称为过零。因此，过零率就是样本改变符号的次数。

定义语音信号 $x_n(m)$ 的短时过零率 Z_n 为

$$Z_n = \frac{1}{2} \sum_{m=0}^{N-1} |\text{sgn}[x_n(m)] - \text{sgn}[x_n(m-1)]| \quad (2)$$

式中， $\text{sgn}[\cdot]$ 是符号函数，即

$$\text{sgn}[x] = \begin{cases} 1, & (x \geq 0) \\ -1, & (x < 0) \end{cases} \quad (3)$$

(3) 双门限法在双门限算法中，短时能量检测可以较好地地区分出浊音和静音。对于清音，由于其能量较小，在短时能量检测中会因为低于能量门限而被误判为静音；短时过零率则可以从语音中区分出静音和清音。将两种检测结合起来，就可以检测出语音段（清音和浊音）及静音段。在基于短时能量和过零率的双门限端点检测算法中首先为短时能量和过零率分别确定两个门限，一个为较低的门限，对信号的变化比较敏感，另一个是较高的门限。当低门限被超过时，很有可能是由于很小的噪声所引起的，未必是语音的开始，当高门限被超过并且在接下来的时间段内一直超过低门限时，则意味着语音信号的开始。双门限法是利用二级判决来实现端点检测的。

1.2.2 双门限法的实现

双门限法进行端点检测步骤如下：

1. 计算信号的短时能量和短时平均过零率。

```
amp=STEn(x,wlen,inc); % 求取短时平均能量
zcr=STZcr(x,wlen,inc); % 计算短时平均过零率
```

这里调用了 STEn 和 STZcr 函数来计算短时能量和短时过零率，这两个函数是自定义的，用于计算每帧的短时能量和过零率。

2. 根据语音能量的轮廓选取一个较高的门限 T_2 ，语音信号的能量包络大部分都在此门限之上，这样可以进行一次初判。语音起止点位于该门限与短时能量包络交点 N_3 和 N_4 所对应的时间间隔之外。
3. 根据背景噪声的能量确定一个较低的门限 T_1 ，并从初判起点往左，从初判终点往右搜索，分别找到能零比曲线第一次与门限 T_1 相交的 2 个点 N_2 和 N_5 ，于是 N_2N_5 段就是用双门限法所判定的语音段。其中，高门限 T_2 相当于代码中的 amp1，低门限 T_1 相当于代码中的 amp2，它用于确定语音段的起始和终止点。

```
% 计算初始无话段区间能量和过零率的平均值
ampth = mean(amp(1:NIS)); % 初始静音段的平均能量
zcrth = mean(zcr(1:NIS)); % 初始静音段的平均过零率
% 设置能量和过零率的阈值
amp2 = 2 * ampth; % 低门限，相当于  $T_1$ 
amp1 = 4 * ampth; % 高门限，相当于  $T_2$ 
zcr2 = 2 * zcrth; % 过零率的门限
```

在参考书的示例中，高门限 amp1 是平均能量的四倍，低门限 amp2 是平均能量的两倍。根据初始静音段的平均能量和过零率设置门限值。这些门限值用于后续的端点检测。

注意，这些参数是需要根据自己的数据的具体特征来更改的，否则无法进行正确的端点检测。具体的调整参数的方法可以参看 2.2。

4. 以短时平均过零率为准，从 N_2 点往左和 N_5 往右搜索，找到短时平均过零率低于某阈值 T_3 的两点 N_1 和 N_6 ，这便是语音段的起止点。

```
xn = 1;
for n = 1:fn
    switch status
    case {0, 1} % 0 = 静音, 1 = 可能开始
        if amp(n) > amp1 % 确信进入语音段
            x1(xn) = max(n - count(xn) - 1, 1);
            status = 2;
            silence(xn) = 0;
            count(xn) = count(xn) + 1;
            % ... ..
        end
    end
end
```

在这段代码中，status 变量用于跟踪当前的检测状态。当 status 为 0 或 1 时，表示当前处于静音或可能开始语音段的状态。如果短时能量 amp(n) 超过高门限 amp1，则认为已经确信进

入语音段，并将 status 更新为 2，表示当前处于语音段状态。代码中的 x1 和 x2 数组用于存储检测到的语音段的起始和终止点。x1(xn) 存储起始点，x2(xn) 存储终止点。

注意：门限值要通过多次实验来确定，门限都是由背景噪声特性确定的。语音起始段的复杂度特征与结束时的有差异，起始时幅度变化比较大，结束时幅度变化比较缓慢。在进行起止点判决前，通常都要采集若干帧背景噪声并计算其短时能量和短时平均过零率，作为选择 M_1 和 M_2 的依据。

1.3 训练 HMM 的过程及其中涉及的三大算法

隐马尔可夫模型 (Hidden Markov Models, 简称为 HMM)，作为语音信号的一种统计模型，在语音处理各个领域特别是语音识别方向获得了广泛的应用。

一个用于语音识别的 HMM 通常用三组模型参数 $M = \{A, B, \pi\}$ 来定义。假设某 HMM 共有 N 个状态 $\{S_i\}_{i=1}^N$ ，那么各参数的定义如下：

- A ：状态转移概率矩阵

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1N} \\ \vdots & \ddots & \vdots \\ a_{N1} & \cdots & a_{NN} \end{bmatrix}$$

其中， a_{ij} 是从状态 S_i 到状态 S_j 转移时的转移概率，且必须满足 $0 \leq a_{ij} \leq 1$ ， $\sum_{j=1}^N a_{ij} = 1$

- π ：系统初始状态概率的集合； $|\pi_i|_{i=1}^N$ 表示初始状态是 s_i 的概率，即，

$$\pi_i = P\{S_1 = s_i\} \quad (1 \leq i \leq N) \quad \sum_{j=1}^N \pi_j = 1$$

- B ：输出观测值概率的集合。 $B = \{b_j(k)\}$ ，其中 b_{ij} 是从状态 S_i 到状态 S_j 转移时观测值 x_i 为符号 k 的输出概率。根据观测集合 X 的取值可将 HMM 分为连续型和离散型 HMM 等。

为了解决识别、寻找与给定观察字符序列对应的最佳的状态序列和模型训练这三个问题，HMM 有对应的三个基本算法。

1.3.1 前向-后向算法

前向-后向算法 (Forward-Backward, 简称为 F-B 算法) 是用来计算给定一个观察值序列 $O = o_1, o_2, \dots, o_T$ 以及一个模型 $M = \{A, B, \pi\}$ 时，由模型 M 产生 O 的概率 $P(O|M)$ 。为有效求出 $P(O|M)$ ，Baum 等人提出前向-后向算法。设 S_1 是初始状态， S_N 是终了状态，则前向-后向算法的描述如下。

(1) 前向算法

前向算法根据输出观察符号序列从前向后递推计算输出概率。前向概率 $\alpha_t(j)$ 由下面的递推公式计算可得：

$$\alpha_0(1) = 1, \quad \alpha_0(j) = 0 \quad (j \neq 1) \quad (4)$$

$$\alpha_t(j) = \sum_i \alpha_{t-1}(i) a_{ij} b_{ij}(o_t) \quad (t = 1, 2, \dots, T; i, j = 1, 2, \dots, N) \quad (5)$$

最后结果

$$P(O|M) = \alpha_T(N) \quad (6)$$

具体步骤如下：

1. 给每个状态准备一个数组变量 $\alpha_t(j)$ ，初始化时令初始状态 S_1 的数组变量 $\alpha_0(1)$ 为 1，其他状态数组变量 $\alpha_0(j)$ 为 0。
2. 根据 t 时刻输出的观察符号 o_t 计算 $\alpha_t(j)$ ：

$$\begin{aligned} \alpha_t(j) &= \sum_i \alpha_{t-1}(i) a_{ij} b_{ij}(o_t) \\ &= \alpha_{t-1}(1) a_{1j} b_{1j}(o_t) \\ &\quad + \alpha_{t-1}(2) a_{2j} b_{2j}(o_t) \\ &\quad + \cdots \\ &\quad + \alpha_{t-1}(N) a_{Nj} b_{Nj}(o_t) \quad (t = 1, 2, \dots, T; j = 1, 2, \dots, N) \end{aligned} \quad (7)$$

当状态 S_i 到状态 S_j 没有转移时， $a_{ij} = 0$ 。

3. 当 $t \neq T$ 时转移到 2)，否则执行 4)。
4. 把最终的数组变量 $\alpha_T(N)$ 内的值取出，则

$$P(O|M) = \alpha_T(N) \quad (8)$$

这种前向递推计算算法的计算量大为减少，变为 $N(N+1)(T-1) + N$ 次乘法和 $N(N-1)(T-1) + N$ 次加法。因此，当 $N = 5$ ， $T = 100$ 时，算法只需大约 3000 次计算（乘法）。另外，这种算法也是一种典型的模型结构，和动态规划（DP）递推方法类似。

(2) 后向算法

与前向算法类似，后向算法即按输出观察值序列的时间，从后向前递推计算输出概率的方法。定义 $\beta_t(i)$ 为后向概率，即从状态 S_i 开始到状态 S_N 结束输出部分符号序列 $o_{t+1}, o_{t+2}, \dots, o_T$ 的概率，则 $\beta_t(i)$ 可由下面的递推公式计算得到：

$$\beta_T(N) = 1, \quad \beta_T(j) = 0 \quad (j \neq N) \quad (9)$$

$$\beta_t(i) = \sum_j \beta_{t+1}(j) a_{ij} b_{ij}(o_{t+1}) \quad (t = T, T-1, \dots, 1; i, j = 1, 2, \dots, N) \quad (10)$$

最后结果

$$P(O|M) = \sum_{i=1}^N \beta_1(i) \pi_i = \beta_0(1) \quad (11)$$

后向算法的计算量大约为 N^2T 数量级，也是一种格型结构。显然，根据定义的前向和后向概率，有如下关系成立：

$$P(O|M) = \sum_{i=1}^N \sum_{j=1}^N \alpha_t(i) a_{ij} b_{ij}(o_{t+1}) \beta_{t+1}(j), \quad 1 \leq t \leq T-1 \quad (12)$$

1.3.2 维特比 Viterbi 算法

第二个要解决的问题是给定观察字符序列和模型 $M = \{A, B, \pi\}$ ，如何有效确定与之对应的最佳的状态序列。这可由另一个 HMM 的基本算法 Viterbi 算法来解决。Viterbi 算法解决了给定一个观察值序列 $O = o_1, o_2, \dots, o_T$ 和一个模型 $M = \{A, B, \pi\}$ 时，在最佳的意义上确定一个状态序列 $S = s_1, s_2, \dots, s_T$ 的问题。此处，最佳意义上的状态序列是指使 $P(S, O/M)$ 最大时确定的状态序列，即 HMM 输出一个观察值序列 $O = o_1, o_2, \dots, o_T$ 时，可能通过的状态序列路径有多种，这里面使输出概率最大的状态序列 $S = s_1, s_2, \dots, s_T$ 就是“最佳”。

Viterbi 函数调用格式：

```
function [prob,q] = viterbi(hmm,0)
```

参数说明：输入参数 **hmm** 为 HMM 结构体，其中保存着 HMM 的模型参数；**0** 为输入语音信号的 MFCC 特征序列 ($T \times D$, T 为帧数, D 为向量维数)。输出参数 **prob** 为输出概率；**q** 为状态序列。

Viterbi 算法可描述如下：

初始化

$$\alpha_0^i(1) = 1, \quad \alpha_0^i(j) = 0 \quad (j \neq 1) \quad (13)$$

递推公式

$$\alpha_t^i(j) = \max_k \{ \alpha_{t-1}^k(i) a_{ij} b_{ij}(o_t) \} \quad (t = 1, 2, \dots, T; i, j = 1, 2, \dots, N) \quad (14)$$

最后结果

$$P_{\max}(S, O/M) = \alpha_T^i(N) \quad (15)$$

在这个递推公式中，每一次使 $\alpha_t^i(j)$ 最大的状态 i 组成的状态序列就是所求的最佳状态序列。利用 Viterbi 算法求取最佳状态序列的步骤如下：

1) 给每个状态准备一个数组变量 $\alpha_t^i(j)$ ，初始化时令初始状态 S_1 的数组变量 $\alpha_0^i(1)$ 为 1，其他状态的数组变量 $\alpha_0^i(j)$ 为 0。

2) 根据 t 时刻输出的观察符号 o_t 计算 $\alpha_t^i(j)$ ：

$$\alpha_t^i(j) = \max_k \{ \alpha_{t-1}^k(i) a_{ij} b_{ij}(o_t) \} \quad (j = 1, 2, \dots, N) \quad (16)$$

$$= \max_k \{ \alpha_{t-1}^k(1) a_{ij} b_{ij}(o_t), \alpha_{t-1}^k(2) a_{ij} b_{ij}(o_t), \dots, \alpha_{t-1}^k(N) a_{ij} b_{ij}(o_t) \} \quad (17)$$

当状态 S_i 到状态 S_j 没有转移时， $a_{ij} = 0$ 。

设计一个符号数组变量，称为最佳状态序列寄存器，利用这个最佳状态序列寄存器把每一次使 $\alpha_t^i(j)$ 最大的状态 i 保存下来。

3) 当 $t \neq T$ 时转移到 2)，否则执行 4)。

4) 把最终的状态寄存器 $\alpha_T^i(N)$ 内的值取出，则 $P_{\max}(S, O/M) = \alpha_T^i(N)$ 为输出最佳状态序列寄存器的值，即为所求的最佳状态序列。

1.3.3 Baum-Welch 算法

M 使 $P(O|M)$ 最大，是一个泛函极值问题。但是，由于给定的训练序列有限，因而不存在一个最佳的方法来估计 M 。此时，Baum-Welch 算法利用递归的思想，使 $P(O|M)$ 局部放大，最后得到优化的模型参数 $M = \{A, B, \pi\}$ 。可以证明，利用 Baum-Welch 算法的重估公式得到的重估模型参

数构成的新模型 $\hat{M}$ ，一定有 $P(O|\hat{M}) > P(O|M)$ 成立，即由重估公式得到的 $\hat{M}$ 比 M 在表示观察值序列 $O = o_1, o_2, \dots, o_T$ 方面更好。重复该过程，逐步改进模型参数，直到 $P(O|\hat{M})$ 收敛，即不再明显增大，此时的 $\hat{M}$ 即为所求模型。

其函数形式为：

```
function hmm = baum_welch(hmm, obs)
```

参数说明：输入参数 **hmm** 为经过初始化的 HMM 结构体，其中保存着 HMM 的模型参数；**obs** 为用于训练的语音信号的 MFCC 特征序列。输出参数 **hmm** 为训练完成后的 HMM 结构体。这个函数通过迭代计算前向和后向概率，重估 HMM 的状态转移概率和观测概率，从而优化模型参数。这个过程重复进行，直到模型收敛或达到预定的迭代次数。

Baum-Welch 算法的步骤如下：

给定一个（训练）观察值符号序列 $O = o_1, o_2, \dots, o_T$ ，以及一个需要通过训练进行重估参数的 HMM 模型 $M = \{A, B, \pi\}$ 。按前向-后向算法，设对于符号序列 $O = o_1, o_2, \dots, o_T$ ，在时刻 t 从状态 S_i 转移到状态 S_j 的转移概率为 $\gamma(i, j)$ ，则 $\gamma(i, j)$ 可表示如下：

$$\gamma_t(i, j) = \frac{\alpha_{t-1}(i)a_{ij}b_j(o_t)\beta_t(j)}{\alpha_t^*(i)} \quad (18)$$

同时，对于符号序列 $O = o_1, o_2, \dots, o_T$ ，在时刻 t 时 Markov 链处于状态 S_i 的概率为

$$\xi_t(i) = \frac{\alpha_t(i)\beta_t(i)}{\alpha_t^*(i)} \quad (19)$$

此时，对于符号序列 $O = o_1, o_2, \dots, o_T$ ，从状态 S_i 转移到状态 S_j 的转移次数的期望值为 $\sum \gamma_t(i, j)$ ；而从状态 S_i 转移出去的次数的期望值为 $\sum \xi_t(i, j)$ 。由此，可得 Baum-Welch 算法中著名的重估公式：

$$a_{ij} = \frac{\sum_t \gamma_t(i, j)}{\sum_t \xi_t(i, j)} \quad (20)$$

$$b_{ij}(k) = \frac{\sum_{t:o_t=k} \gamma_t(i, j)}{\sum_t \gamma_t(i, j)} \quad (21)$$

所以根据观察值序列 $O = o_1, o_2, \dots, o_T$ ，和选取的初始模型 $\hat{M} = \{\hat{A}, \hat{B}, \hat{\pi}\}$ ，由重估公式，求得一组新参数 $\hat{a}_{ij}$ 和 $\hat{b}_{ij}(k)$ ，就得到了一个新的模型 $\hat{M} = \{\hat{A}, \hat{B}, \hat{\pi}\}$ 。

在代码实现中，**hmm.trans(i, j)** 表示 a_{ij} ，实现过程为：

```
%---重估转移概率矩阵A
for i=1:N-1
    demon=0;
    for k=1:K
        tmp=param(k).ksai(:,i,:);
        demon=demon+sum(tmp(:)); %对时间t, j求和
    end
    for j=i:i+1
        nom=0;
        for k=1:K
            tmp=param(k).ksai(:,i,j);
            nom=nom+sum(tmp(:)); %对时间t求和
        end
        hmm.trans(i,j)=nom/demon;
```

```

end
end

```

在代码实现中, `hmm.mix(j).weight(l)` 表示 b_{ij} , 实现过程为:

```

%----重估输出观测值概率B
for j=1:N %状态循环
    for l=1:hmm.M(j) %混合高斯的数目
        %计算各混合成分的均值和协方差矩阵
        nommean=zeros(1,SIZE);
        nomvar=zeros(1,SIZE);
        denom=0;
        for k=1:K %训练数目的循环
            T=size(obs(k).fea,1); %帧数
            for t=1:T %帧数(时间)的遍历
                x=obs(k).fea(t,:);
                nommean=nommean+param(k).gama(t,j,l)*x;
                nomvar=nomvar+param(k).gama(t,j,l)*(x-mix(j).mean(1,:)).^2;
                denom=denom+param(k).gama(t,j,l);
            end
        end
        hmm.mix(j).mean(l,:)=nommean/denom;
        hmm.mix(j).var(l,:)=nomvar/denom;

        %计算各混合成分的权重
        nom=0;
        denom=0;
        for k=1:K
            tmp=param(k).gama(:,j,l);
            nom=nom+sum(tmp(:));
            tmp=param(k).gama(:,j,:);
            denom=denom+sum(tmp(:));
        end
        hmm.mix(j).weight(l)=nom/denom;
    end
end
end

```

下面给出利用 Baum-Welch 算法进行 HMM 训练的具体步骤:

1) 适当地选择 a_{ij} 和 $b_{ij}(k)$ 的初始值。常用的 π 的设定方式为:

$$a_{ij} = \frac{1}{\text{从状态 } i \text{ 转移出去的弧的条数}} \quad (22)$$

给予每一个输出观察符号相同的输出概率初始值, 即

$$b_{ij}(k) = \frac{1}{\text{码本中码字的个数}} \quad (23)$$

并且每条弧上给予相同的输出概率矩阵。

2) 给定一个(训练)观察值符号序列 $O = o_1, o_2, \dots, o_T$, 由初始模型计算 $\gamma_t(i, j)$ 等, 并且由重估公式, 计算 $\hat{a}_{ij}$ 和 $\hat{b}_{ij}(k)$ 。

3) 再给定一个 (训练) 观察值符号序列 $O = o_1, o_2, \dots, o_T$, 把前一次的 $\hat{a}_{ij}$ 和 $\hat{b}_{ij}(k)$ 作为初始模型计算 $\gamma_t(i, j)$ 等, 由重估公式, 重新计算 $\hat{a}_{ij}$ 和 $\hat{b}_{ij}(k)$ 。

4) 如有必要, 重复 2) 和 3), 直到 $\hat{a}_{ij}$ 和 $\hat{b}_{ij}(k)$ 收敛为止。

需要注意的是, 语音识别一般采用从左到右型 HMM, 所以初始状态概率不需要估计, 一般设定为

$$\pi_i = 1, \pi_j = 0, (i = 2, \dots, N) \quad (24)$$

模型收敛, 停止训练的判定方法也很重要。因为并不是训练次数越多, 训练过头反而会使模型参数精度变差。一种判定方法是前后两次的输出概率的差值小于一定限值或模型参数几乎不变为止; 另一种判定方法是采用固定训练次数的办法, 如对于一定数量的训练数据, 利用这些数据反复训练十次 (或者下次) 即可。另外, 训练数据的数量也很重要, 一般来讲, 要想训练一个好的 HMM, 至少需要同类训练数据几千个左右。

2 数据采集与预处理

2.1 语音采集

在本设计中采集了三个不同语音数据, 命名为 Jan, Feb, Mar, 对这三条数据进行语音的读入, 并绘制音频信号波形图。

```
wlen = round(0.025 * fs); % 帧长为25毫秒
inc = round(0.02 * fs); % 帧移为20毫秒
amp2 = 70 * ampth;
amp1 = 100 * ampth; % 设置能量和过零率的阈值
zcr2 = 50 * zcrth;
```

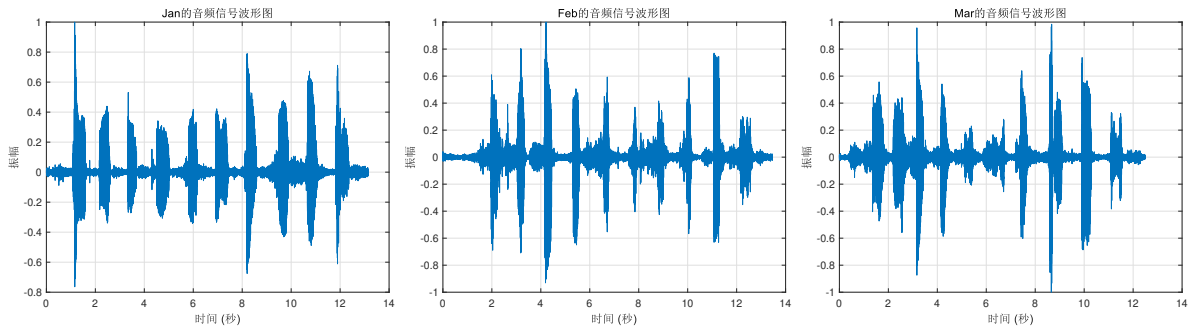


图 2: 不同语音信号的波形图

```
clc;clear;close all;
[y, fs] = audioread('Jan.wav');
noverlap = 128;
window = hamming(256); % 汉明窗
nfft = 512;
spectrogram(y, window, noverlap, nfft, fs, 'yaxis');title('Jan的音频信号频谱图');
print('Jan_sp','-depsc','-painters');
```

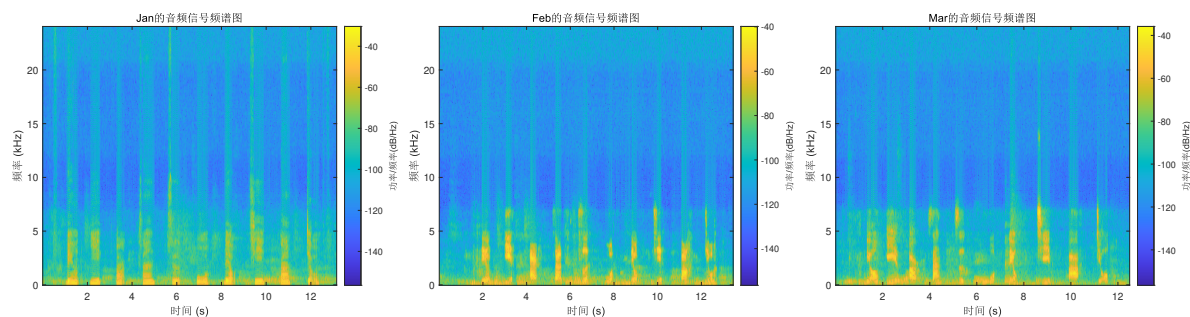


图 3: 不同语音信号的频谱图

2.2 端点检测

在本设计中，用双门限法实现端点检测。结合教材所给代码，可得端点检测的效果为：

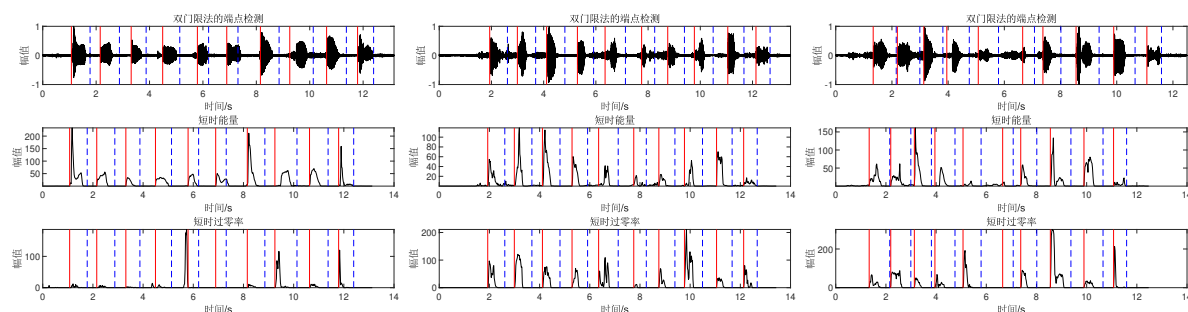


图 4: 双门限法对不同语音信号端点检测

在本实验中，应该注意，由于三条语音中，间隔都比较高，所以需要即使调整静音段长度和能量阈值，如果直接用课本上参考的参数和阈值，就会有很多个端点。

对于不同语音，应该注意以下特征：

1. 语音段和静音段的区分：阈值需要足够区分语音段和静音段。通常，静音段的能量和过零率较低，而语音段的能量和过零率较高。
2. 数字之间的停顿：由于每个数字之间都有停顿，这些停顿可能会被错误地识别为语音段的结束或开始。因此，阈值需要足够高，以避免这种情况。
3. 信号的动态范围：语音信号的能量和过零率可能会有较大的动态范围。阈值需要设置在一个合理的范围内，以适应这种动态变化。
4. 实验调整：可能需要通过实验来确定最佳的阈值。可以从一些初始值开始，然后根据端点检测的结果进行调整。

在本实验中，参数如下：

```
wlen = round(0.025 * fs); % 帧长为 25 毫秒
inc = round(0.02 * fs); % 帧移为 20 毫秒
amp2 = 70 * ampth;
amp1 = 100 * ampth; % 设置能量和过零率的阈值
zcr2 = 50 * zcrth;
```

在进行端点检测时，说话者并不是 0-9 依次录制，得到 $3 * 9 = 27$ 段数据，而是从 0-9 连续的说话，只有 3 段完整的数据，所以进行端点检测时，可以得到每个数字的的区间和其范围，样本索引由下表所示。

注意，在训练的过程中，样本索引的区间对于改代码非常有帮助，在改代码的时候输出索引的区间，就可以方便的定位具体的数字，可以判断代码逻辑的正确性，所以列在下表。

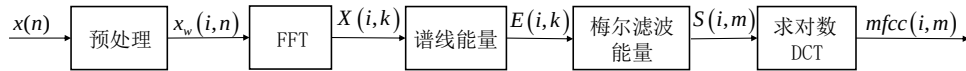
表 1: 说话人每个孤立字的索引长度

索引长度	数字 0	数字 1	数字 2	数字 3	数字 4	数字 5	数字 6	数字 7	数字 8	数字 9
说话人 1	417	429	333	381	249	249	417	525	441	357
说话人 2	405	417	417	369	465	297	381	429	369	321
说话人 3	489	477	405	477	429	249	381	489	453	309

2.3 特征提取

2.3.1 MFCC 特征提取方法介绍

MFCC 特征参数提取原理框图如下：



预处理

预处理包括预加重、分帧、加窗。

预加重

预加重的目的是为了补偿高频分量的损失，提升高频分量。预加重的滤波器可设为

$$H(z) = 1 - az^{-1} \quad (25)$$

式中， a 为一个常数，如 0.97。

分帧处理

通常语音信号是准稳态的信号，但是将它分为较短的帧后，每帧信号可以当作稳态信号，从而用处理稳态信号的方法来处理。同时，为了使一帧与另一帧之间的参数能较平稳地过渡，因此相邻帧之间会进行部分重叠处理。

加窗

加窗：加窗的目的是减少频域中的泄漏，将每一帧语音乘以汉明窗或海宁窗。语音信号 $x(n)$ 经预处理后为 $x_i(m)$ ，其中下标 i 表示分帧后的第 i 帧。

快速傅里叶变换

对每一帧信号进行 FFT 变换，从时域数据转变为频域数据

$$X(i, k) = \text{FFT}[x_i(m)] \quad (26)$$

计算谱线能量

计算每一帧 FFT 后的数据谱线能量：

$$E(i, k) = |X_i(k)|^2 \quad (27)$$

计算美尔滤波器能量

在频域中相当于把每帧谱线能量谱通过美尔滤波器，并计算在该美尔滤波器中的能量。在频域中相当于把每帧谱线能量谱 $E(i, k)$ （其中 i 表示第 i 帧， k 表示频域中的第 k 条谱线）与美尔滤波器的频域响应 $H_m(k)$ 相乘并相加

$$S(i, m) = \sum_{k=0}^M E(i, k) H_m(k), \quad 0 \leq m \leq M \quad (28)$$

计算 $x(n)$ 的 FFT 倒谱

求取 FFT 的倒谱是 $X(k)$ 取对数后计算的逆变换。这里求 DCT 的倒谱和求 FFT 的倒谱相似，把美尔滤波器的能量取对数后计算 DCT

$$mf(cc(i, n)) = \sqrt{\frac{2}{M}} \sum_{m=0}^{M-1} \ln |S(i, m)| \cos \left[\frac{\pi n(2m-1)}{2M} \right] \quad (29)$$

式中， $S(i, m)$ 是求出的美尔滤波器能量； m 是指第 m 个美尔滤波器（共有 M 个）； i 是指第 i 帧； n 是 DCT 后的谱线。

2.3.2 代码实现

教材中附带的mfcc.m函数，在使用的过程中很容易出现问题。调用时发现，

```
dtm=zeros(size(m)); %差分系数
```

此处经常报错：m 没有定义，经过仔细的调试，我发现mfcc.m函数中总是会跳过下面这个循环：

```
%计算每帧的MFCC参数
for i=1:size(xx,1)
    y=xx(i,:);
    s=y'.*hamming(256);
    t=abs(fft(s));
    t=t.^2;
    c1=log(bank*t(1:129));
    c1=dctcoef*c1;
    c2=c1.*w';
    m(i,:)=c2';
end
```

仔细研究了 10.1 章节的内容，发现mfcc.m函数中，传的参数 x 并不是端点提取中得到的 voiceseg 帧区间，而是索引，所以需要把 voiceseg 映射到索引处。

```
function cc=mfcc(x)
```

这里很容易被误导的地方在于，教材中给出的 tra_data.mat 中每个数据的大小都是小于 100，很容易与 voiceseg 对应，但是 voiceseg 只是帧的值，实际语音的索引大小分别是：631552，545888，

600064, 而 voiceseg 的代表帧数, 所以比较小, 此时在mfcc.m函数中, size(xx,1) 小于 1, 就会跳过 for 循环中的内容, 会出现 m 没有定义的报错。

对于以下端点检测得到的 voiceseg, 还需要进行一个映射关系:

% 端点检测函数

```
[voiceseg1, ~, ~, ~, amp1, zcr1] = vad_TwoThr(speechIn1, wlen1, inc1, NIS1);
[voiceseg2, ~, ~, ~, amp2, zcr2] = vad_TwoThr(speechIn2, wlen1, inc1, NIS1);
[voiceseg3, ~, ~, ~, amp3, zcr3] = vad_TwoThr(speechIn3, wlen1, inc1, NIS1);
```

映射的代码如下: 因为只有三个样本数据, 所以写循环反而更麻烦, 直接更改变量名称即可。

```
Segments1 = struct('startIdx', {}, 'endIdx', {});
for i = 1:length(voiceseg1)
    % 将帧索引转换为样本索引
    Segments1(i).startIdx = fix(frameTime1(voiceseg1(i).begin) * FS1);
    Segments1(i).endIdx = fix(frameTime1(voiceseg1(i).end) * FS1);
end
Segments2 = struct('startIdx', {}, 'endIdx', {});
for i = 1:length(voiceseg2)
    % 将帧索引转换为样本索引
    Segments2(i).startIdx = fix(frameTime2(voiceseg2(i).begin) * FS2);
    Segments2(i).endIdx = fix(frameTime2(voiceseg2(i).end) * FS2);
end
Segments3 = struct('startIdx', {}, 'endIdx', {});
for i = 1:length(voiceseg3)
    % 将帧索引转换为样本索引
    Segments3(i).startIdx = fix(frameTime3(voiceseg3(i).begin) * FS3);
    Segments3(i).endIdx = fix(frameTime3(voiceseg3(i).end) * FS3);
end
```

只要注意到以上细节, 对每条语音调用 obs(i).fea = mfcc(segment), 就可以将每个孤立字的特征存储在 obs(i).fea 中。

```
for i = 1:length(voiceseg1)
    segment = speechIn1(Segments1(i).startIdx:Segments1(i).endIdx);
    obs(i).sph = segment;
    obs(i).fea = mfcc(segment);
    obs(i).segment = segment;
    speechSegments(1, i) = obs(i);
end
% ... 另外三个数据 ...
```

注意在以上内容中, speechSegments 是一个二维的结构体, speechSegments(i, j) 代表第 i 个说话人的第 j-1 个数字的语音。例如 speechSegments(2, 10) 代表第 2 个说话人的第 9 个数字的语音。

3 实验分析

3.1 多说话人实验

在模仿教材所给代码的基础上，我首先完成了多说话人的实验。所以在本报告中，我先介绍多说话人的实验过程，然后再介绍单说话人的实验过程。

3.1.1 多说话人的 HMM 训练

在初始化隐马尔可夫模型时，obs 变量用于存储上一节中提取的特征值，因此之前的 sph 和 segment 变量均不再需要。

```
function hmm = inithmm(obs, N, M) % 初始化 HMM 模型
```

使用 Baum-Welch 算法训练 HMM。

```
N = 4; % 状态数
M = [3, 3, 3, 3]; % 每个状态的混合模型成分数
% 初始化 HMM
hmm = struct('N', {}, 'M', {}, 'init', {}, 'trans', {}, 'mix', {});
% 训练 HMM
for d = 1:10 % 遍历每个数字
    for p = 1:3 % 遍历每个人
        fprintf('\n训练数字%d的hmm\n', d);
        hmm_temp = inithmm(speechSegments(p, d), N, M); % 初始化 HMM模型
        hmm{d} = baum_welch(hmm_temp, speechSegments(p, d)); % 迭代更新 HMM 的各参数
    end
end
```

训练结果：

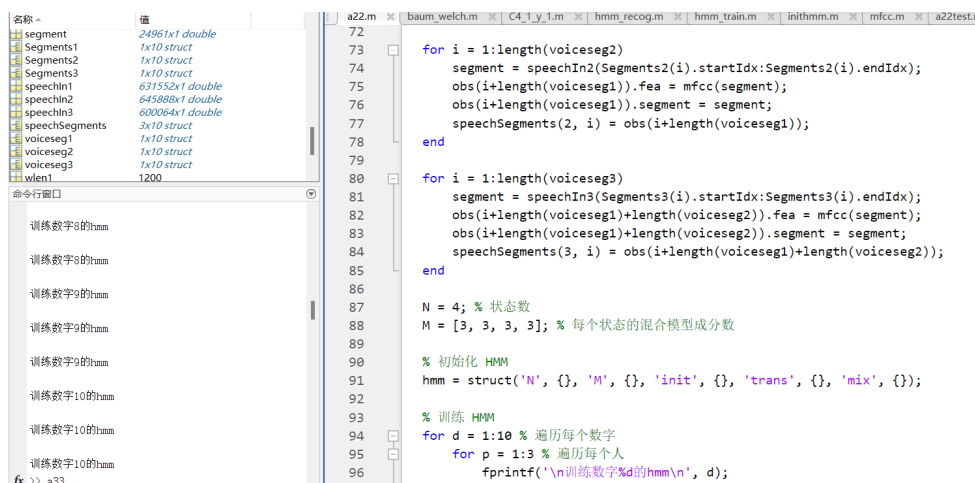


图 6: 多说话人 HMM 训练完成截图

因为有三组数据，在循环中每个分别遍历，每个数字的三个数据共同训练 hmmd，实现对 0-9 十个数字的训练。

3.1.2 多说话人的 HMM 测试

完成训练之后，测试的实现就比较简单， k 表示第几个说话人， j 表示数字，提取当前语音的特征，计算当前语音关于各数字 hmm 的 $p(X|M)$ ，概率最大的就是识别结果，可以看到，可以得到较为准确的识别结果。

```
fprintf('开始识别\n');
k = 1; %第几个人
j = 5; %数字
rec_sph=speechSegments(k,j).segment; % 随机选择一条待识别语音
fprintf('该语音的真实值为%d\n',j);
rec_fea = mfcc(rec_sph); % 特征提取

% 求出当前语音关于各数字hmm的p(X|M)
for i=1:10
    pxsm(i) = viterbi(hmm{i}, rec_fea);
end
[d,n] = max(pxsm); % 判决，将该最大值对应的序号作为识别结果
fprintf('该语音识别结果为%d\n',n)
```

测试结果：

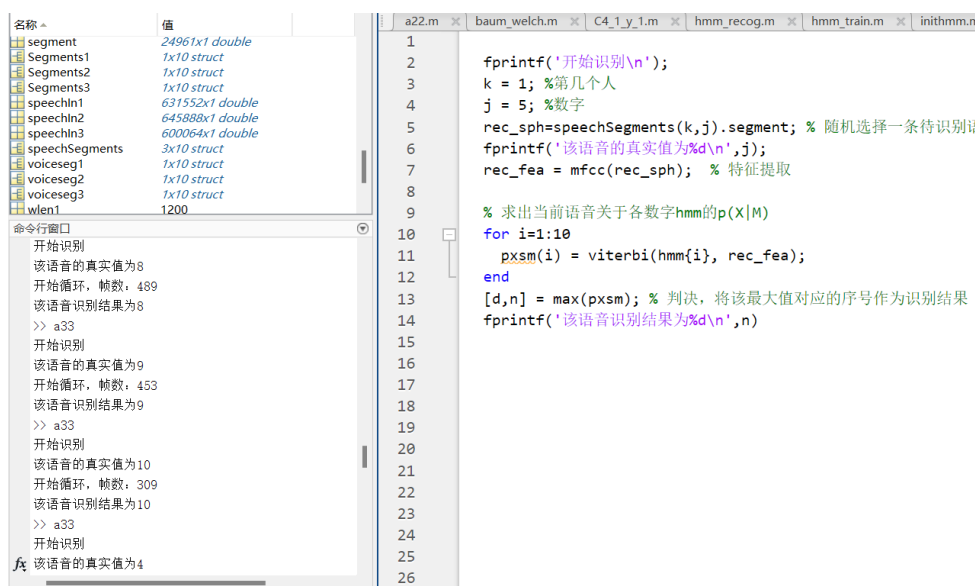


图 7: 多说话人 HMM 测试结果

说话人 1-3 都作为训练集时测试集的准确率为：50%。

说话人 1-1 都作为训练集时测试集（说话人 3）的准确率为：30%。

可以看到在多说话人实验中，孤立字识别的效果并不好。

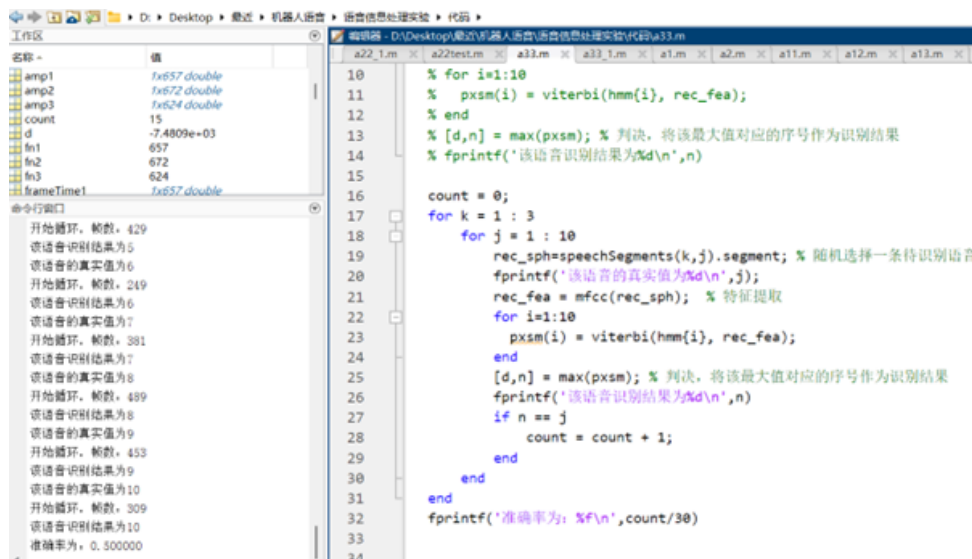


图 8: 说话人 1-3 作为训练集

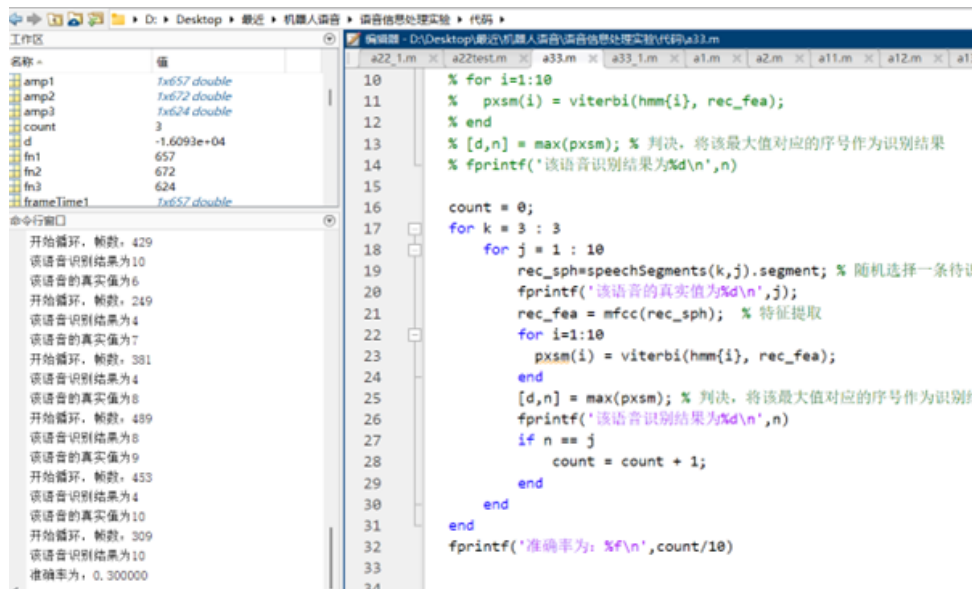


图 9: 说话人 1-2 作为训练集

3.2 单说话人实验

3.2.1 单说话人的 HMM 训练

在之前多说话人已经完成的基础上，只需要在循环中修改遍历的人数，就可以改成单说话人。以下代码介绍了第二个说话人的训练过程。

```
% 训练 HMM
for d = 1:10 % 遍历每个数字
    for p = 2:2 % 第二个说话人
        fprintf('\n训练数字%d的hmm\n', d);
        hmm_temp = inithmm(speechSegments(p, d), N, M); % 初始化 HMM模型
        hmm{d} = baum_welch(hmm_temp, speechSegments(p, d)); % 迭代更新 HMM的各参数
    end
end
end
```

下图展示了单说话人的训练过程。

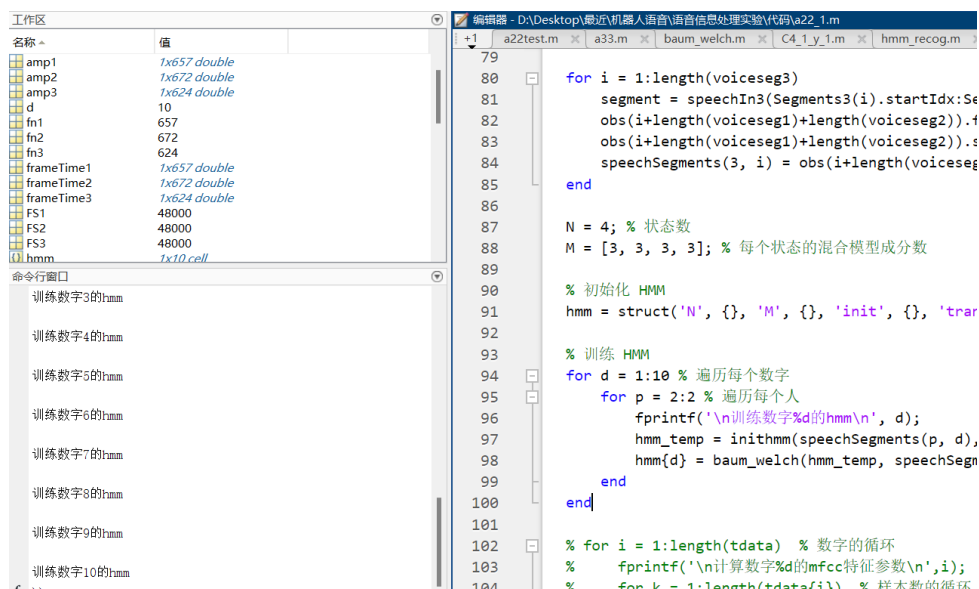


图 10: 单说话人 HMM 训练结果

3.2.2 单说话人的 HMM 测试

在多说话人的测试的基础上，单说话人测试中只需要固定 k 值即可，如以下实验，固定第二个说话人。

```
fprintf('开始识别\n');
k = 2; %第几个人
j = 9; %数字
rec_sph=speechSegments(k,j).segment; % 随机选择一条待识别语音
fprintf('该语音的真实值为%d\n',j);
rec_fea = mfcc(rec_sph); % 特征提取
```

```
% 求出当前语音关于各数字hmm的p(X|M)
for i=1:10
    pxsm(i) = viterbi(hmm{i}, rec_fea);
end
[d,n] = max(pxsm); % 判决, 将该最大值对应的序号作为识别结果
fprintf('该语音识别结果为%d\n',n)
```

测试结果如下图, 可以看到, 能很好的完成单说话人的孤立字识别。同时可以看到, 在训练和测试的过程中, 帧数对应的序列索引是非常有用的, 因为不同孤立字的序列索引一般不同, 在训练和测试的过程中显示序列索引, 就知道自己的 `speechSegments(k,j)` 是否与实际中的第 `k` 个人的第 `j-1` 个数对应, 从而方便代码的修改, 也能验证代码的正确性。

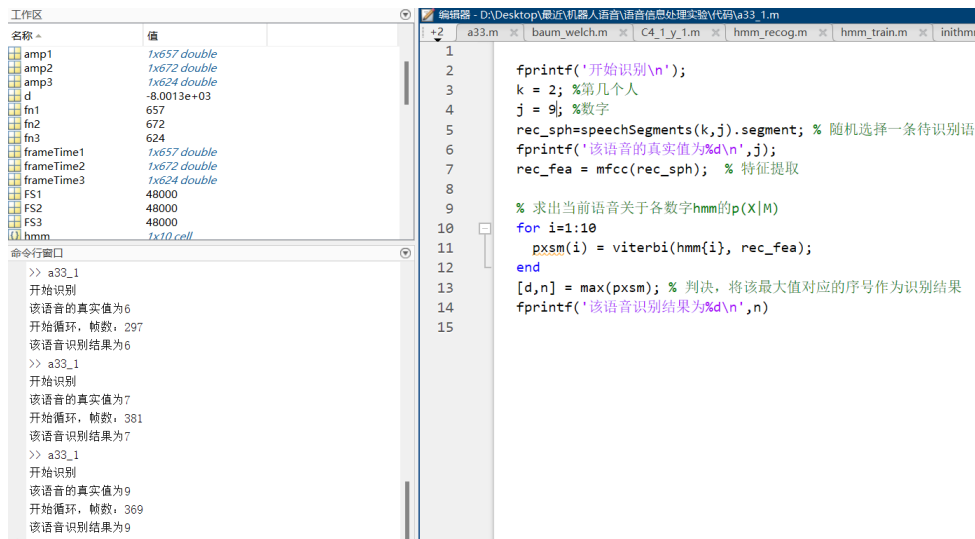


图 11: 单说话人 HMM 测试结果

3.3 HMM 状态数的影响

以单说话人实验为例, 当参数如下所示时:

```
N = 4; % 状态数
M = [3, 3, 3, 3]; % 每个状态的混合模型成分数
```

统一在测试中识别数字 8, 保持混合模型分数均为 3, 测试 `N` 改变时的概率, 比较识别精度和计算成本。

用 matlab 的 `tic`, `toc` 可以计算不同 `N` 下的 HMM 训练过程中的运行时间, 代码如下

```
tic;
N = 6; % 状态数
M = [3, 3, 3, 3, 3, 3]; % 每个状态的混合模型成分数
% 初始化 HMM
hmm = struct('N', {}, 'M', {}, 'init', {}, 'trans', {}, 'mix', {});
% 训练 HMM
for d = 1:10 % 遍历每个数字
    for p = 2:2 % 遍历每个人
```

```

fprintf('\n训练数字%d的hmm\n', d);
hmm_temp = inithmm(speechSegments(p, d), N, M); % 初始化 HMM模型
hmm{d} = baum_welch(hmm_temp, speechSegments(p, d)); % 迭代更新 HMM的各
    参数
end
end
toc;
fprintf('运行时间: %f 秒\n', toc);

```

实验结果如表3所示:

表 2: 比较 N 值对 HMM 训练的影响

各数字 $P(X M)$	N=1	N=2	N=4	N=6
数字 0	-16675.5755	-18033.34595	-22789.24409	-21724.15197
数字 1	-18713.87774	-21363.76629	-29222.60966	-30270.02826
数字 2	-13766.9012	-14734.78246	-16935.96352	-16751.60243
数字 3	-13536.24414	-12817.86953	-14163.99042	-15639.9603
数字 4	-17710.34194	-18988.26048	-19049.6519	-19221.34376
数字 5	-17220.91946	-17598.49202	-19873.734	-19443.12965
数字 6	-16800.3272	-16089.55604	-18815.82071	-18096.02085
数字 7	-21438.06275	-20094.75209	-21327.86319	-26052.88677
数字 8	-10061.56993	-8748.207131	-8001.288572	-7474.877475
数字 9	-15621.57888	-15640.89299	-17417.15341	-15593.78545
训练时间 (s)	4.834404	13.357496	36.108439	109.742037

根据表格中数据可得, 在 $N = 1, 2, 4, 6$ 时, 对于测试数字 8, HMM 都可以正确识别, 但是随着 N 的增大, 训练过程的运行时间显著增大, 可见状态数影响:

1. 模型复杂度: 增加状态数可以提供更复杂的模型, 能够捕捉更细微的序列特征。
2. 计算成本: 状态数的增加会导致计算成本显著增加, 特别是在进行 Baum-Welch 算法时, 需要计算的状态转移概率和观测概率。

同时, 状态数的改变还会影响准确率, 用以下代码生成在改变状态数分别为: 1, 2, 4, 6 时的准确率。

```

count = 0;
for k = 3 : 3
    for j = 1 : 10
        rec_sph=speechSegments(k,j).segment; % 随机选择一条待识别语音
        fprintf('该语音的真实值为%d\n',j);
        rec_fea = mfcc(rec_sph); % 特征提取
        for i=1:10
            pxsm(i) = viterbi(hmm{i}, rec_fea);
        end
        [d,n] = max(pxsm); % 判决, 将该最大值对应的序号作为识别结果
    end
end

```

```

fprintf('该语音识别结果为%d\n',n)
if n == j
    count = count + 1;
end
end
end
fprintf('准确率为: %f\n',count/10)

```

实验数据如下表所示：

表 3: 比较 N 值对 HMM 训练准确率的影响

说话人	N=1	N=2	N=4	N=6
Jan	100%	100%	100%	100%
Feb	100%	100%	100%	90%
Mar	100%	100%	100%	90%

当 N 为 6 时，Feb 和 Mar 说话人的检测准确率降低到 0.9，说明 N 的增大会导致模型过拟合，从而降低准确率

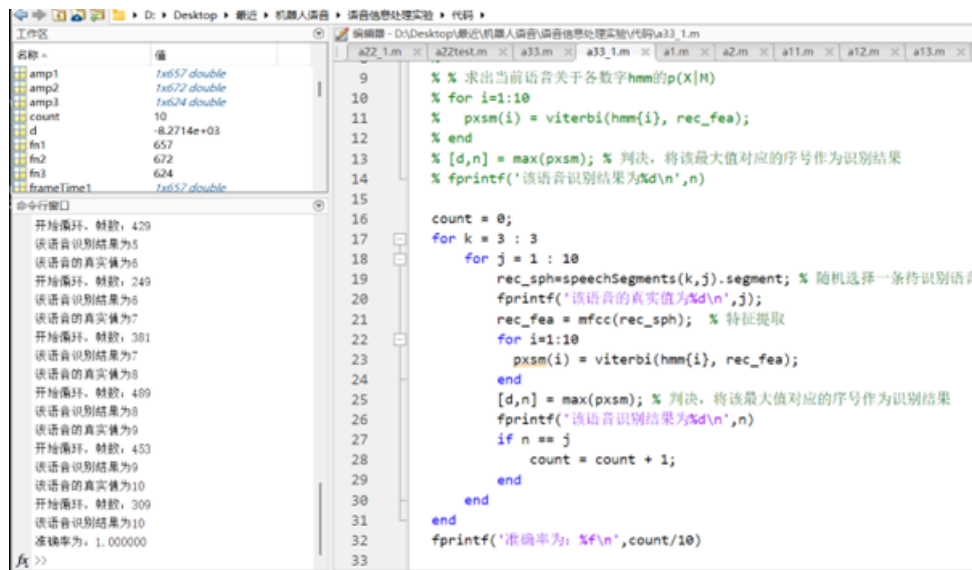


图 12: N=2 时说话人 Mar 的测试结果

由于孤立字的识别比较简单，样本也比较少，所以当 N 改变时，都可以准确识别，但是在某些情况下，增加状态数可以提高模型的识别精度，因为它能够更细致地区分序列中的不同部分。然而，更多的状态也意味着模型更复杂，需要更多的数据来避免过拟合。

3.4 高斯混合成分数的影响

以单说话人实验为例，控制 N=4 不变，统一在测试中识别数字 8，记录改变 M 时各数字 $P(X|M)$ ，比较识别精度和计算成本。实验结果如表 4 所示。

表 4: 比较 M 值对 HMM 训练的影响

各数字 $P(X M)$	M=1	M=2	M=3	N=4	N=6
数字 0	-21403.98233	-22029.77233	-22789.24409	-23411.07601	-23724.51475
数字 1	-19650.32761	-22374.54419	-29222.60966	-27716.7068	-29107.25016
数字 2	-13790.3572	-15168.90669	-16935.96352	-16555.34008	-16047.55964
数字 3	-13237.76135	-13735.21532	-14163.99042	-14457.0081	-15467.4945
数字 4	-16483.97836	-18172.83332	-19049.6519	-19589.50337	-20597.92723
数字 5	-16811.36401	-18653.27435	-19873.734	-20665.56707	-21175.08384
数字 6	-15428.01243	-18054.65319	-18815.82071	-18665.44206	-19086.35836
数字 7	-18235.57288	-21931.19918	-21327.86319	-26802.13311	-25889.41742
数字 8	-9137.902744	-8413.596194	-8001.288572	-7651.974297	-7189.498789
数字 9	-14611.37915	-16784.22244	-17417.15341	-17023.51502	-17903.37985
训练时间 (s)	18.921164	27.646469	36.108439	70.568620	84.274583

每个状态的混合模型成分数 (M) 决定了该状态由多少个高斯混合成分组成。这些高斯成分用于模拟观测数据的生成过程。对高斯混合分数进行类似于状态数的实验, 发现状态分数也会影响准确率。根据表格中数据可得, 在 $M = 1, 2, 3, 4, 6$ 时, 对于测试数字 8, HMM 都可以正确识别, 但是随着 M 的增大, 训练过程的运行时间显著增大, 可见混合模型成分数影响参数数量和识别性能。

参数数量: 混合模型成分数的增加会导致模型参数数量的增加, 需要更多的数据来估计这些参数。识别性能: 在适当的上下文中, 增加混合模型成分数可以提高识别性能, 因为它允许模型更细致地模拟观测数据的分布。

在实际应用中, 可能还需要考虑其他因素, 如数据集的大小、特征的类型和特定任务的需求。此外, 现代的语音识别系统可能使用深度学习方法, 如深度神经网络 (DNN), 这些方法在某些情况下可以提供比传统 HMM 更好的性能。

4 总结

在本次实验中, 我先完成了对三个说话人 0-9 孤立字语音的采集和端点检测, 并分别完成了多说话人和单说话人人的 HMM 训练与测试, 并在单说话人的基础上分别分析了 HMM 状态数 N 与高斯混合分数 M 对识别结果的影响。

在语音采集中, 在实验过程中发现教材中所给函数已经无法识别, 查阅相关资料发现: 在新的版本中, 以及无法识别相关的函数。在 MATLAB 的早期版本中, 语音信号的录制和处理主要依赖于 wavrecord、wavread 和 wavwrite 等函数。然而, 从 MATLAB 2015 版本开始, 这些函数已经被更新或替代, 以提供更强大的功能和更好的兼容性²。

在端点检测过程中, 由于采集到的录音中, 每个数字的间隔较大, 教材中的参数设置不能很好的检测采集到的录音中的端点, 通过不断的调整参数, 可以实现较准确的语音端点检测。

对 MFCC 算法的正确理解, 直接影响映射 voiceseg 到索引: 在将端点检测的结果 voiceseg 映射到音频信号的索引时, 遇到了总是无法进入循环的困难。由于 voiceseg 存储的是帧数, 而不是样本索引, 所以需要将这帧数转换为样本索引。这一步骤在实现时较为复杂, 需要仔细处理以确保每个语音段正确对应。

在实现过程中，代码调试是一个反复的过程。特别是在处理三维数据结构体 `speechSegments` 时花费了大量时间确保每个语音段的起始和结束索引正确无误。此外，也遇到了一些运行时错误，如变量未定义或数组维度不匹配等，这些都需要逐步调试和对代码的修正。

在实验中，使用了三段不同说话人的音频文件。但是在处理不同采样率的音频文件中，没有用循环和函数，这就导致我的代码牵一发而动全身，一些变量的命名有很多重复，整体有较大冗余，而且不利于封装，但是优点在于，逻辑比较清晰，且易于编写。

在分析 HMM 状态数 N 与高斯混合分数 M 对识别结果的影响时，我在单说话人基础上，分别设置 $N = 1, 2, 4, 6$ 和 $M = 1, 2, 3, 4, 6$ ，记录每个孤立字的条件概率与计算时间和准确率，发现 N 和 M 会影响模型复杂度和计算成本，同时也对识别的准确率有影响。

通过这次实验，我深入理解了语音信号的特征提取过程，特别是在处理实际音频数据时遇到的挑战。**未来学习过程中不求一切顺利，但期待每一次的峰回路转。**

参考文献

- [1] 梁瑞宇, 赵力, and 魏昕. 语音信号处理实验教程. *Experimental course on speech signal processing*. 高等院校通信与信息专业规划教材. 北京: 机械工业出版社, 2016, xi, 289 页. ISBN: 978-7-111-53071-8.
- [2] Aly El-Osery. “MATLAB Tutorial”. In: *October..* <http://www.ee.nmt.edu> (2004).

A 附录：代码

实验环境：win11, matlab2021b

语音信号的采集：注意，因为采集语音时，不可能三个说话人都在电脑旁边采集，所以我是以微信语音的形式采集的，将 wav 文件上传到电脑，此时在代码中只需要实现读入操作即可。

```
clc;clear;close all;
% 读取音频文件
[y_1, fs_1] = audioread('Jan.wav');
ymax_1 = max(abs(y_1));
y_1 = y_1 / ymax_1;
% 绘制音频信号波形图
t_1 = (1:length(y_1)) / fs_1;
% subplot(1,3,1);
plot(t_1, y_1);
xlabel('时间 (秒)');
ylabel('振幅');
title('Jan的音频信号波形图');
grid on;
print('Jan', '-depsc', '-painters');
```

端点检测：

```
[x,fs]=audioread('Jan.wav'); % 读入数据文件
x=x/max(abs(x)); % 幅度归一化
N=length(x); % 取信号长度
time=(0:N-1)/fs; % 计算时间

subplot 311
plot(time,x,'k');
title('双门限法的端点检测');
ylabel('幅值'); axis([0 max(time) -1 1]);
xlabel('时间/s');

% wlen=200; inc=80; % 分帧参数
wlen = round(0.025 * fs); % 帧长为25毫秒
inc = round(0.02 * fs); % 帧移为12.5毫秒
IS=0.1; overlap=wlen-inc; % 设置IS
NIS=fix((IS*fs-wlen)/inc +1); % 计算NIS
fn=fix((N-wlen)/inc)+1; % 求帧数
frameTime=FrameTimeC(fn, wlen, inc, fs);% 计算每帧对应的时间

[voiceseg,vsl,SF,NF,amp,zcr]=vad_TwoThr(x,wlen,inc,NIS); % 端点检测

subplot 312
plot(frameTime,amp,'k');
ylim([min(amp) max(amp)])
title('短时能量');
```

```

ylabel('幅值');
xlabel('时间/s');
subplot 313
plot(frameTime,zcr,'k');
ylim([min(zcr) max(zcr)])
title('短时过零率');
ylabel('幅值');
xlabel('时间/s');

disp(voiceseq);

for k=1 : vsl % 画出起止点位置
    subplot 311
    nx1=voiceseq(k).begin; nx2=voiceseq(k).end;
    nx1=voiceseq(k).duration;
    line([frameTime(nx1) frameTime(nx1)],[-1.5 1.5],'color','r','LineStyle','-');
    line([frameTime(nx2) frameTime(nx2)],[-1.5 1.5],'color','b','LineStyle','--');
    subplot 312
    line([frameTime(nx1) frameTime(nx1)],[min(amp) max(amp)],'color','r','LineStyle',
        '-');
    line([frameTime(nx2) frameTime(nx2)],[min(amp) max(amp)],'color','b','LineStyle',
        '--');
    subplot 313
    line([frameTime(nx1) frameTime(nx1)],[min(zcr) max(zcr)],'color','r','LineStyle',
        '-');
    line([frameTime(nx2) frameTime(nx2)],[min(zcr) max(zcr)],'color','b','LineStyle',
        '--');
end

```

多说话人 HMM 训练代码：在此代码中，用到了很多重复的操作，这是因为只有三条语音，写循环反而会冗余且不清晰，增加了代码的考量。

```

clc;clear;close all;

% 开！！让大局逆转吧！！

[speechIn1,FS1] = audioread('Jan.wav');
[speechIn2,FS2] = audioread('Feb.wav');
[speechIn3,FS3] = audioread('Mar.wav');

speechIn1 = speechIn1 / max(abs(speechIn1)); % 幅度归一化
speechIn2 = speechIn2 / max(abs(speechIn2)); % 幅度归一化
speechIn3 = speechIn3 / max(abs(speechIn3)); % 幅度归一化

wlen1 = round(0.025 * FS1); % 帧长为25毫秒
inc1 = round(0.02 * FS1); % 帧移为12.5毫秒
IS=0.1; overlap=wlen1-inc1; % 设置IS
NIS1=fix((IS*FS1-wlen1)/inc1 +1); % 计算NIS

```

```

wlen2 = round(0.025 * FS2); % 帧长为25毫秒
inc2 = round(0.02 * FS2); % 帧移为12.5毫秒
IS=0.1; overlap=wlen2-inc2; % 设置IS
NIS2=fix((IS*FS2-wlen2)/inc2 +1); % 计算NIS

wlen3 = round(0.025 * FS3); % 帧长为25毫秒
inc3 = round(0.02 * FS3); % 帧移为12.5毫秒
IS=0.1; overlap=wlen1-inc1; % 设置IS
NIS3=fix((IS*FS3-wlen3)/inc3 +1); % 计算NIS

fn1 = fix((length(speechIn1) - wlen1) / inc1) + 1; % 求帧数
fn2 = fix((length(speechIn2) - wlen1) / inc1) + 1;
fn3 = fix((length(speechIn3) - wlen1) / inc1) + 1;

% 计算每帧对应的时间
frameTime1=FrameTimeC(fn1, wlen1, inc1, FS1);
frameTime2=FrameTimeC(fn2, wlen2, inc2, FS2);
frameTime3=FrameTimeC(fn3, wlen3, inc3, FS3);

% 端点检测函数
[voiceseg1, ~, ~, ~, amp1, zcr1] = vad_TwoThr(speechIn1, wlen1, inc1, NIS1);
[voiceseg2, ~, ~, ~, amp2, zcr2] = vad_TwoThr(speechIn2, wlen1, inc1, NIS1);
[voiceseg3, ~, ~, ~, amp3, zcr3] = vad_TwoThr(speechIn3, wlen1, inc1, NIS1);

Segments1 = struct('startIdx', {}, 'endIdx', {});
for i = 1:length(voiceseg1)
    % 将帧索引转换为样本索引
    Segments1(i).startIdx = fix(frameTime1(voiceseg1(i).begin) * FS1);
    Segments1(i).endIdx = fix(frameTime1(voiceseg1(i).end) * FS1);
end
Segments2 = struct('startIdx', {}, 'endIdx', {});
for i = 1:length(voiceseg2)
    % 将帧索引转换为样本索引
    Segments2(i).startIdx = fix(frameTime2(voiceseg2(i).begin) * FS2);
    Segments2(i).endIdx = fix(frameTime2(voiceseg2(i).end) * FS2);
end
Segments3 = struct('startIdx', {}, 'endIdx', {});
for i = 1:length(voiceseg3)
    % 将帧索引转换为样本索引
    Segments3(i).startIdx = fix(frameTime3(voiceseg3(i).begin) * FS3);
    Segments3(i).endIdx = fix(frameTime3(voiceseg3(i).end) * FS3);
end

% 初始化 obs 结构体数组
obs = struct('fea', {}, 'segment', {});
for i = 1:length(voiceseg1)

```

```

        segment = speechIn1(Segments1(i).startIdx:Segments1(i).endIdx);
        obs(i).sph = segment;
        obs(i).fea = mfcc(segment);
        obs(i).segment = segment;
        speechSegments(1, i) = obs(i);
    end

    for i = 1:length(voicseg2)
        segment = speechIn2(Segments2(i).startIdx:Segments2(i).endIdx);
        obs(i+length(voicseg1)).fea = mfcc(segment);
        obs(i+length(voicseg1)).segment = segment;
        speechSegments(2, i) = obs(i+length(voicseg1));
    end

    for i = 1:length(voicseg3)
        segment = speechIn3(Segments3(i).startIdx:Segments3(i).endIdx);
        obs(i+length(voicseg1)+length(voicseg2)).fea = mfcc(segment);
        obs(i+length(voicseg1)+length(voicseg2)).segment = segment;
        speechSegments(3, i) = obs(i+length(voicseg1)+length(voicseg2));
    end

%比较参数对HMM的影响时，只需要更改N，M两个参数即可

N = 4; % 状态数
M = [3, 3, 3, 3]; % 每个状态的混合模型成分数

% 初始化 HMM
hmm = struct('N', {}, 'M', {}, 'init', {}, 'trans', {}, 'mix', {});
tic;
% 训练 HMM
for d = 1:10 % 遍历每个数字
    for p = 1:3 % 遍历每个人
        fprintf('\n训练数字%d的hmm\n', d);
        hmm_temp = inithmm(speechSegments(p, d), N, M); % 初始化 HMM模型
        hmm{d} = baum_welch(hmm_temp, speechSegments(p, d)); % 迭代更新 HMM的各参数
    end
end
toc;
fprintf('运行时间: %f 秒\n', toc);

```

单说话人 HMM 测试代码：只需要在多说话人的基础上，更改 p 即可，这也是我先完成多说话人的原因，因为多说话人的逻辑很容易应用到单说话人。

```

for d = 1:10 % 遍历每个数字
    for p = 2:2 % 遍历每个人
        fprintf('\n训练数字%d的hmm\n', d);
        hmm_temp = inithmm(speechSegments(p, d), N, M); % 初始化 HMM模型
        hmm{d} = baum_welch(hmm_temp, speechSegments(p, d)); % 迭代更新 HMM的各参数
    end
end

```

```
end
```

HMM 的测试：多说话人和单说话人代码相同，只是注意，单说话人时， k 只能取之前给的 p 值，而多说话人 k 可以取 1-3.

```
fprintf('开始识别\n');
k = 2; %第几个人
j = 9; %数字
rec_sph=speechSegments(k,j).segment; % 随机选择一条待识别语音
fprintf('该语音的真实值为%d\n',j);
rec_fea = mfcc(rec_sph); % 特征提取

% 求出当前语音关于各数字hmm的p(X|M)
for i=1:10
    pxsm(i) = viterbi(hmm{i}, rec_fea);
end
[d,n] = max(pxsm); % 判决，将该最大值对应的序号作为识别结果
fprintf('该语音识别结果为%d\n',n)
```

B 附录：端点检测部分的参数设置

在本实验中，我遇到的难点主要有两个，一个是端点检测时，我采集的语音很难正确采集到端点，这需要多次的调整，二是 HMM 训练的过程，受教材附带代码的误导，一开始一直在找 `redata.mat` 文件或者试图生成 `redata.mat` 文件，后来才发现：与其尝试生成 `mat` 文件，不如自己按照原代码的逻辑直接自己写，写的过程遇到帧数的问题，但是逻辑清晰后就比较容易。

所以在附录中，记录自己设置参数的过程以及部分思考。

当参数设置为教材参考参数时：

```
wlen=200; inc=80; % 分帧参数
amp2=2*ampth;
amp1=4*ampth; % 设置能量和过零率的阈值
zcr2=2*zcrth;
```

结果如图13所示：

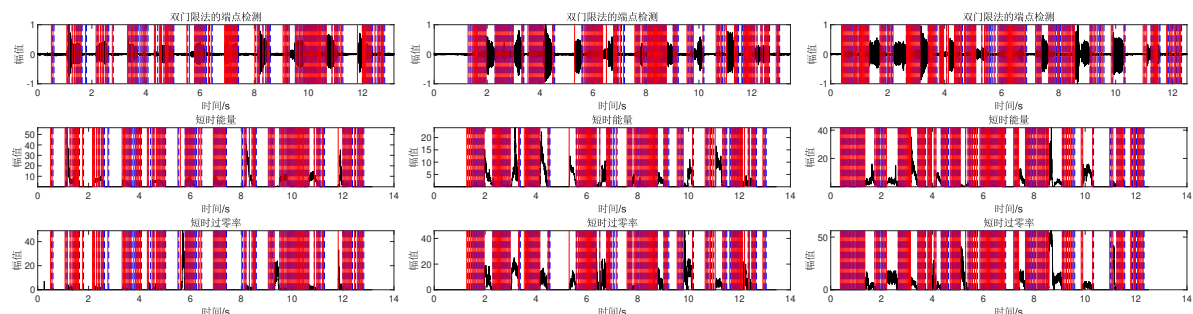


图 13: 教材上的参数对应的端点检测结果

可以注意到，虽然每段有用数字都被划分出来，但是划分的段太多了，`Mar.wav` 的 10 个孤立字被划分了 179 段，这显然是不利于之后的 HMM 测试的。

考虑到采集到的语音间隔比较大，所以先确定了帧长，参数如下：

```
wlen = round(0.025 * fs); % 帧长为25毫秒
inc = round(0.02 * fs); % 帧移为20毫秒
```

增加帧长后集结果如14所示，可以看到，已经显著的接近的目标值，10 个孤立字几乎都被划分为 15 段左右。

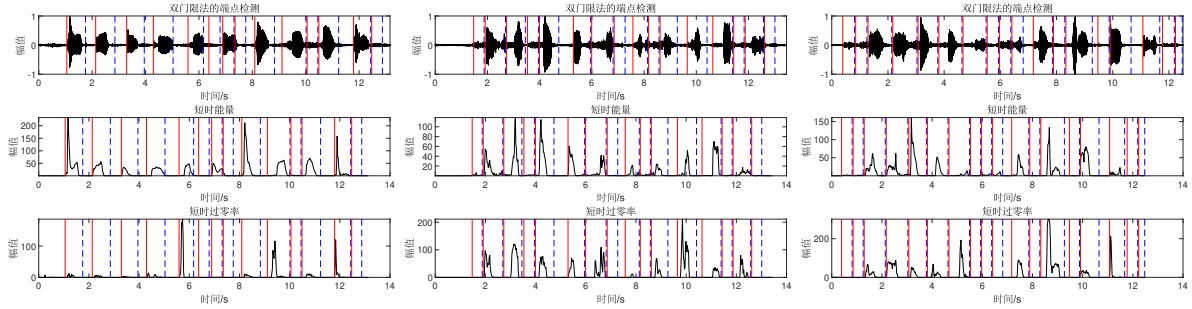


图 14: 改变帧长对应的端点检测结果

之后就是不厌其烦的修改能量和过零率的阈值，只要增加 amp2, amp1, zcr2, 总会出现三条语音同时被检测成功的情况，这是因为三条语音都是在几乎 12 秒的时间内说完了 10 个孤立字，其间隔时间和频率相差不大。

```
amp2=40*ampth;
amp1=60*ampth; % 设置能量和过零率的阈值
zcr2=50*zcrth;
```

使用以上参数，端点检测的结果如15所示，这里可以看出，处理 Jan.wav 之外，其他两条语音均正确采集 10 个端点，只是 Mar.wav 语音由于 4, 5 数字那里说话声音比较小，所以端点划分不太准确，此时需要进一步调整。

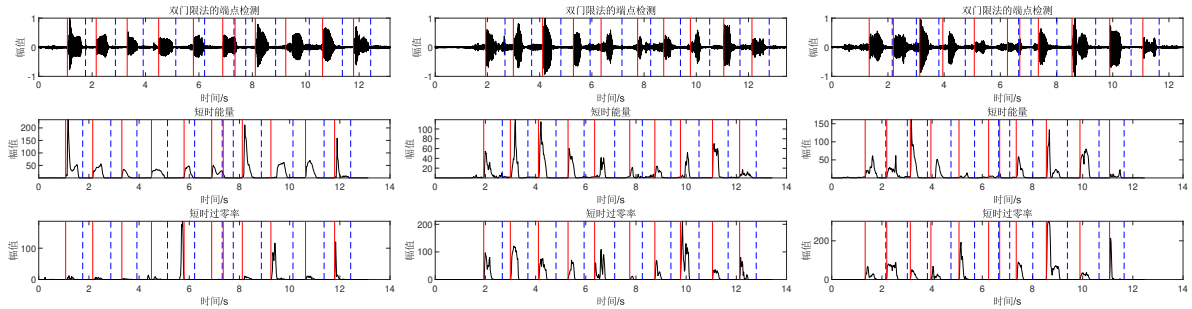


图 15: 调整能量和过零率对应的端点检测结果

经过多次调整，就可以得到正文中2.2所示的结果。参数设置为：

```
wlen = round(0.025 * fs); % 帧长为25毫秒
inc = round(0.02 * fs); % 帧移为20毫秒
amp2 = 70 * ampth;
amp1 = 100 * ampth; % 设置能量和过零率的阈值
zcr2 = 50 * zcrth;
```

使用以上参数，端点检测的结果如图16所示：

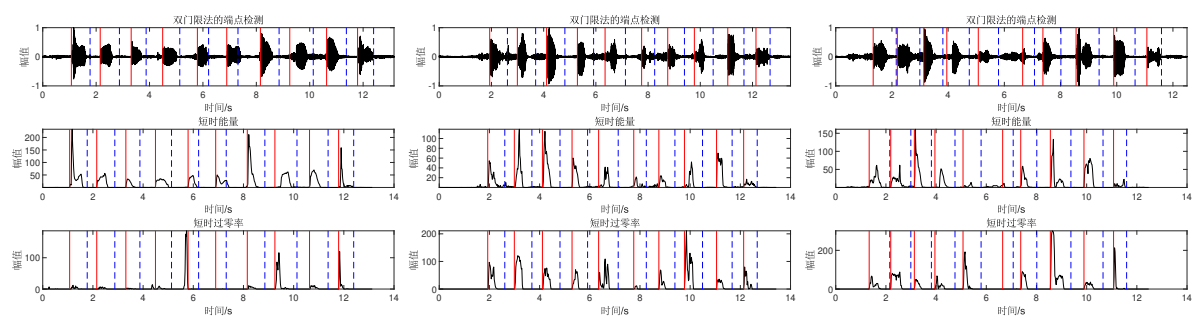


图 16: 调整能量和过零率对应的端点检测结果

在实践中验证真知，在失败中迭代真理。

以上！